



BICOL UNIVERSITY COLLEGE OF SCIENCE
LEGAZPI CITY, ALBAY



IT Elect 1 – WEB DEVELOPMENT

DOCUMENTATION

(Laboratory 7, POPULATING FROM A DATABASE)

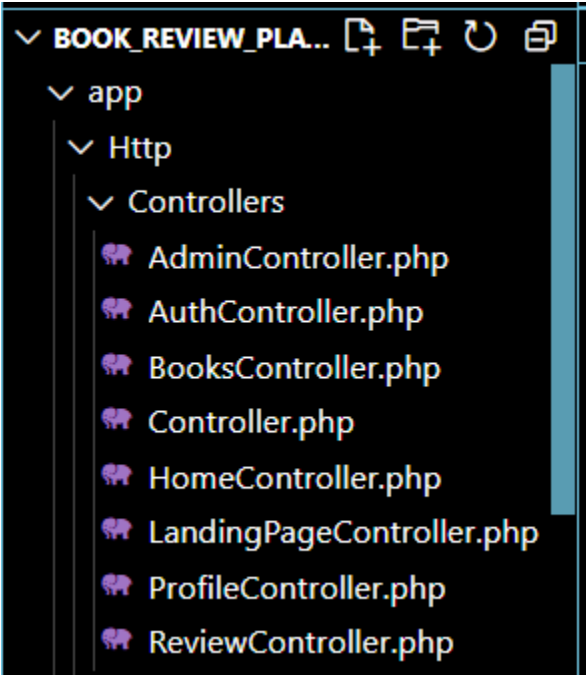
SUBMITTED BY:

JERALD JAY G. BUBAN

BSIT – 3C

Part 1: Create a database and create a connection to it.

Controllers



- These are the controllers we used in this project. They help keep things organized so the website is easier to work with.

NOTE: I won't be putting the complete code because some of it are long, so I'll just put some parts of it.

AdminController

```
app > Http > Controllers > AdminController.php > AdminController > adminBookSearch()
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6  use App\Models\Book;
7  use App\Models\User;
8  use App\Models\Review;
9  use Illuminate\Support\Facades\Log;
10 use Illuminate\Support\Facades\Hash;
11
12 17 references | 0 implementations
13 class AdminController extends Controller
14 {
15     1 reference | 0 overrides
16     public function showBooksDashboard(Request $request): View
17     {
18         $books = Book::all();
19
20         // Return the view with books data and set the active sidebar
21         return view('admin-books-dashboard', data: [
22             'activeSidebar' => 'books',
23             'books' => $books
24         ]);
25     }
26
27     1 reference | 0 overrides
28     public function store(Request $request): RedirectResponse
29     {
30         // Validate the request
31         $validatedData = $request->validate(rules: [
32             'title' => 'required|string|max:255',
33             'author' => 'required|string|max:255',
34             'genres' => 'required|string',
35             'descriptionInput' => 'required|string',
36             'coverImage' => 'required|image|mimes:jpeg,png,jpg,gif,svg|max:2048', // Validate image file
37         ]);
```

- The **AdminController** in this project manages various admin tasks such as for books, users, and reviews. It handles actions like displaying dashboards, searching, adding, updating, or deleting records, and processing file uploads. It also includes functionality for highlighting inappropriate content, managing violations, and supporting AJAX requests for dynamic updates. Essentially, it organizes and processes the admin side of the website, connecting the database, views, and user inputs efficiently.

AuthController

```
app > Http > Controllers > AuthController.php > AuthController > register()
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\User;
6  use Illuminate\Http\Request;
7  use Illuminate\Support\Facades\Auth;
8  use Illuminate\Support\Facades\Hash;
9
10 5 references | 0 implementations
11 class AuthController extends Controller
12 {
13     1 reference | 0 overrides
14     public function login(): View
15     {
16         return view(view: 'auth.login');
17     }
18
19     1 reference | 0 overrides
20     function loginPost(Request $request): mixed|RedirectResponse
21     {
22         $request->validate(rules: [
23             "email" => 'required',
24             "password" => 'required',
25         ]);
26
27         $credentials = $request->only(keys: "email", "password");
28
29         if (Auth::attempt(credentials: $credentials)) {
30             if (Auth::user()->role === 'admin') {
31                 return redirect()->route(route: 'admin.books.dashboard');
32             }
33
34             return redirect()->route(route: 'profile', parameters: ['email' => Auth::user()->email]);
35         }
36
37         return redirect(to: route(name: "login"))->with(key: "error", value: "Login failed");
38     }
39
40     1 reference | 0 overrides
41     function register(): View
42     {
43         return view(view: 'auth.register');
44     }
45
46     1 reference | 0 overrides
47     function registerPost(Request $request): RedirectResponse
48     {
49         $request->validate(rules: [
50             "fullname" => 'required',
51             "email" => 'required',
52             "password" => 'required',
53         ]);
54
55         $user = new User();
56         $user->name = $request->fullname;
57         $user->email = $request->email;
58         $user->password = Hash::make(value: $request->password);
59
60         if ($user->save()) {
61             return redirect(to: route(name: "login"))->with(key: "success", value: "User created successfully");
62         }
63         return redirect(to: route(name: "register"))->with(key: "error", value: "Failed to create account");
64     }
65 }
66
67 }
```

- The **AuthController** manages user authentication and registration. It has methods for displaying login and registration pages, handling user login and registration submissions, and redirecting users based on their roles or login status. It also validates user inputs and uses hashed passwords for security, providing feedback on successful or failed actions.

BooksController

```
app > Http > Controllers > BooksController.php > BooksController > showByReleaseDate()
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6  use App\Models\Book;
7  use Illuminate\Support\Facades\Log;
8  use Illuminate\Support\Facades\Auth;
9  use Illuminate\Support\Facades\Hash;
10
11
12  12 references | 0 implementations
13  class BooksController extends Controller
14  {
15
16      1 reference | 0 overrides
17      public function show($id): View
18      {
19          $book = Book::find(id: $id);
20
21          if (!$book) {
22              abort(code: 404);
23          }
24
25          // Split the genre string into an array to handle multiple genres
26          $bookGenres = explode(separator: ',', string: $book->genre); // Split the stored genre string into an array
27
28          return view(view: 'books.book-details', data: compact(var_name: 'book', var_names: 'bookGenres'));
29      }
30
31      1 reference | 0 overrides
32      public function showByRating(): View
33      {
34          $booksByRating = Book::withAvg(relation: 'reviews', column: 'rating') // Calculate average rating
35              ->orderByDesc(column: 'reviews_avg_rating') // Order by rating
36              ->get();
37      }
38  }
```

- The **BooksController** handles various actions related to books, such as showing book details, browsing by genre or rating, recommending similar books, searching by title or author, and organizing books by release date. It also allows users to toggle favorite books and provides views for displaying results. This focuses on retrieving, organizing, and presenting book data efficiently while logging actions for debugging purposes.

ProfileController:

```
app > Http > Controllers > ProfileController.php > ProfileController > favoriteGenre()
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Http\Controllers\BooksController;
6  use Illuminate\Support\Facades\Log;
7  use Illuminate\Http\Request;
8  use App\Models\Book;
9  use App\Models\Review;
10 use App\Models\User;
11
12  2 references | 0 implementations
13  class ProfileController extends Controller
14  {
15
16      1 reference | 0 overrides
17      public function showProfile(): View
18      {
19          $user = auth()->user(); // Get the currently authenticated user
20
21          $favoriteGenres = $this->favoriteGenre(); // Get the favorite genres
22          $topReviewedBooks = $this->topReviewedBooks(); // Get the top reviewed books
23          $topFavoriteBooks = $this->topFavoriteBooks(); // Get the top 5 most recently added favorite books
24          $reviewedBooksWithAvgRating = $this->getReviewedBooks(user: $user);
25          $favoriteBooks = $this->getFavoriteBooks(user: $user); // Get the favorite books
26
27          $reviews = $user->reviews()->with(relations: 'book')->latest()->get(); // Get the latest 5 reviews with associated books
28
29          return view(view: 'profile', data: [
30              'favoriteGenres' => $favoriteGenres,
31              'topReviewedBooks' => $topReviewedBooks,
32              'topFavoriteBooks' => $topFavoriteBooks, // Add the top 5 favorite books to the view
33              'reviews' => $reviews, // Pass the reviews data to the view
34              'reviewedBooksWithAvgRating' => $reviewedBooksWithAvgRating, // Pass all reviewed books to the view
35              'favoriteBooks' => $favoriteBooks, // Pass favorite books to the view
36          ]);
37      }
38  }
```

- The **ProfileController** manages user profile-related features, including displaying the user's favorite genres, top-rated and favorite books, recently reviewed books, and reviews. It also calculates average ratings for books and organizes data for easy display in the user's profile.

- The goal is to provide a personalized and detailed view of the user's reading preferences and activities.

ReviewController

```
app > Http > Controllers > ReviewController.php > ...
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6  use App\Models\Review;
7  use App\Models\Book;
8  use App\Models\Reply;
9  use Illuminate\Support\Facades\Log;
10 use Illuminate\Support\Facades\Auth;
11
12
13
14 class ReviewController extends Controller
15 {
16     /**
17      * Display a listing of the resource.
18      */
19     public function index(): void
20     {
21         //
22     }
23
24     /**
25      * Show the form for creating a new resource.
26      */
27     public function create(): void
28     {
29         //
30     }
31
32     /**
33
34     /**
```

```
62
63     1 reference | 0 overrides
64     public function showRatings($id): View
65     {
66         $book = Book::find(id: $id);
67
68         $bookGenre = $book ? $book->genre : 'Unknown';
69
70         $bookGenres = explode(separator: ',', string: $bookGenre); // Split the genres into an array
71
72         $reviews = Review::where(column: 'bookID', operator: $id)
73         | | | | | ->with(relations: 'replies') // Load replies for each review
74         | | | | | ->get();
75
76         $ratings = Review::where(column: 'bookID', operator: $id)->pluck(column: 'rating');
77
78         $averageRating = $ratings->avg();
79
80         $totalReviews = $ratings->count();
81
82         $starRatings = [];
83         for ($i = 5; $i >= 1; $i--) {
84             $count = $ratings->filter(callback: fn($rating): bool => $rating == $i)->count();
85             $percentage = $totalReviews > 0 ? ($count / $totalReviews) * 100 : 0;
86             $starRatings[$i] = round(num: $percentage, precision: 1);
87         }
88
89         return view(view: 'books.book-details', data: compact(var_name: 'reviews', var_names: 'starRatings', 'totalReviews', 'averageRating'
90     }
91
92
```

- The **ReviewController** handles everything related to book reviews, including creating, updating, deleting, and displaying reviews. It allows users to add ratings and comments for books, view detailed ratings and reviews, and reply to others' reviews. It also ensures that users can edit or delete their reviews.

Registering Controllers to Routes

```
routes > web.php
1 <?php
2
3 use App\Http\Controllers\AdminController;
4 use App\Http\Controllers\BooksController;
5 use App\Http\Middleware\LogOutOnLandingPage;
6 use Illuminate\Support\Facades\Route;
7 use App\Http\Controllers\HomeController;
8 use App\Http\Controllers\AuthController;
9 use App\Http\Controllers\LandingPageController;
10 use App\Http\Controllers\ReviewController;
11 use App\Http\Controllers\ProfileController;
12
13 Route::get(uri: '/', action: [LandingPageController::class, 'index'])->name(name: 'landing-page');
14
15 // Route for searching books (users)
16 Route::get(uri: '/search', action: [BooksController::class, 'search'])->name(name: 'books.search');
17
18
19 // Authentication Routes (only for guests)
20 Route::middleware(middleware: 'guest')->group(callback: function (): void {
21     Route::get(uri: '/login', action: [AuthController::class, 'login'])->name(name: 'login');
22     Route::post(uri: '/login', action: [AuthController::class, 'loginPost'])->name(name: 'login.post');
23     Route::get(uri: '/register', action: [AuthController::class, 'register'])->name(name: 'register');
24     Route::post(uri: '/register', action: [AuthController::class, 'registerPost'])->name(name: 'register.post');
25 });
26
27
28 // Authenticated Routes (for logged-in users)
29 Route::middleware(middleware: 'auth')->group(callback: function (): void {
30     Route::get(uri: '/contact', action: function (): View {
31         return view(view: 'contact');
32     });
33
34     Route::get(uri: '/home', action: [HomeController::class, 'index'])->name(name: 'home.index');
35     Route::get(uri: '/reviews', action: [ReviewController::class, 'show']);
36
37     Route::post(uri: '/logout', action: function (): RedirectResponse {
38         auth()->logout(); // Logs out the user
39         return redirect(to: route(name: 'landing-page'));
40     })->name(name: 'logout');
41
42     // Content loading routes
43     Route::get(uri: '/home/content', action: [BooksController::class, 'loadBooks'])->name(name: 'home.books');
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
```

- These routes manage the website's navigation and access. The landing page and book search are open to everyone, while login and registration are available only for guests. Logged-in users can access their home page, reviews, and contact page, and they can also log out. Additional routes handle loading book content dynamically for the home page.

```
46 Route::get(uri: '/browse', action: [BooksController::class, 'showByGenre'])->name(name: 'books.byGenre');
47 //will trigger the showByGenre method ng BookController, yung showByGenre will fetch all the genres first (parang naka group by para one instance)
48 //after that it will pass variable sa method niya tas yung unique genres nasa taas parang category (referring sa browse view)
49 //then each genre naka loop siya will display the books related dun sa genre (limit 4)
50
51
52 Route::get(uri: '/browse-books/{genre}', action: [BooksController::class, 'showGenre'])->name(name: 'books.browse.genre');
53 //yung category sa browse view pag clinick yung genre it will show all the books to that genre (view rendered is show-books-by-genre)
54
55 Route::get(uri: '/browse-books/view-all/{genre}', action: [BooksController::class, 'viewAllByGenre'])->name(name: 'view-all.genre.show');
56
57 Route::get(uri: '/browse/books/view-all/release-date/{genre}', action: [BooksController::class, 'showByReleaseDate'])->name(name: 'view-all.release-date.show');
58 // nakabase to sa released at column sa books since latest books siya, see logic sa method niya
59
60 Route::get(uri: '/browse/books/view-all/rating/{genre}', action: [BooksController::class, 'showByRating'])->name(name: 'view-all.rating.show');
61
62 Route::get(uri: '/browse-books/book-detail/{id}', action: [BooksController::class, 'showBookDetail'])->name(name: 'books.bookDetail');
63
64 Route::post(uri: '/browse-books/{id}/reply/{review_id}', action: [ReviewController::class, 'addReply'])->name(name: 'reviews.reply');
65
66 Route::post(uri: '/book/{id}/review', action: [ReviewController::class, 'store'])->name(name: 'reviews.store');
67
68 Route::put(uri: '/reviews/{reviewID}/update', action: [ReviewController::class, 'update'])->name(name: 'reviews.update');
69
70 Route::delete(uri: '/reviews/{reviewID}', action: [ReviewController::class, 'delete'])->name(name: 'reviews.delete');
71
72 Route::get(uri: '/profile', action: [ProfileController::class, 'showProfile'])->name(name: 'profile');
73
74 Route::post(uri: '/favorite/{bookId}/toggle', action: [BooksController::class, 'toggleFavorite'])->name(name: 'favorite.toggle');
75 });
76
```

- These routes handle book browsing, reviews, profiles, and favorites. Users can explore books by genre, release date, or rating, view detailed book pages, and interact with reviews by adding, updating, or deleting them. They can also toggle favorite books and view their profiles.

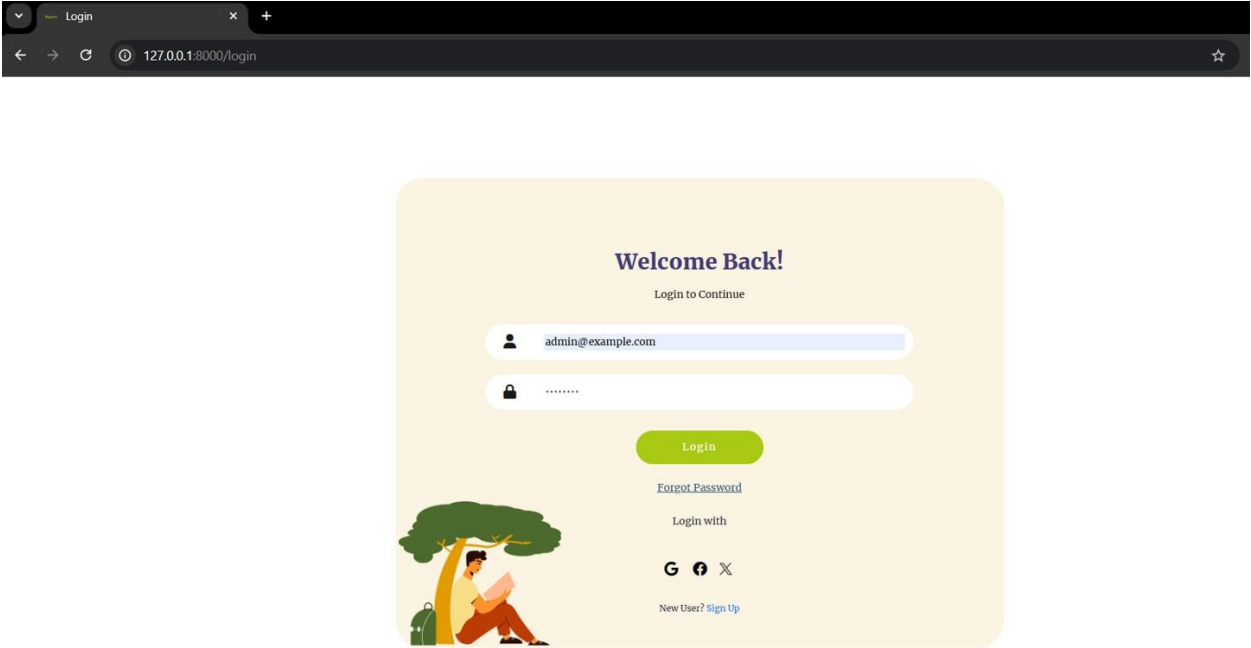
```
77 // Authentication Routes (only for admin)
78 Route::middleware(middleware: ['auth', 'admin'])->group(callback: function (): void {
79     // Show dashboard views
80     Route::get(uri: '/admin-books-dashboard', action: [AdminController::class, 'showBooksDashboard'])->name(name: 'admin.books.dashboard');
81     Route::get(uri: '/admin/admin-users-dashboard', action: [AdminController::class, 'showUsersDashboard'])->name(name: 'admin.users.dashboard');
82     Route::get(uri: '/admin/admin-reviews-dashboard', action: [AdminController::class, 'showReviewsDashboard'])->name(name: 'admin.reviews.dashboard');
83
84     // Routes for Admin User Management
85     Route::get(uri: '/admin/users/search', action: [AdminController::class, 'adminUserSearch'])->name(name: 'admin.users.search');
86     Route::delete(uri: '/admin/users/delete/{id}', action: [AdminController::class, 'deleteUser'])->name(name: 'admin.users.delete');
87     Route::get(uri: '/admin/users/show-violators', action: [AdminController::class, 'showUsersWithViolations'])->name(name: 'admin.users.showViolato
88
89     //Routes for Admin Review Management
90     Route::get(uri: '/admin/reviews/search', action: [AdminController::class, 'adminReviewSearch'])->name(name: 'admin.reviews.search');
91     Route::post(uri: '/admin/reviews/delete', action: [AdminController::class, 'adminDeleteReviews'])->name(name: 'admin.reviews.delete');
92     Route::post(uri: '/admin/review/{userID}/violation', action: [AdminController::class, 'incrementViolation'])->name(name: 'admin.reviews.violatio
93
94     //Routes for Admin Book Management
95     Route::get(uri: '/admin/books/search', action: [AdminController::class, 'adminBookSearch'])->name(name: 'admin.books.search'); // Route for sear
96     Route::get(uri: '/admin/books/search/genre', action: [AdminController::class, 'adminSearchByGenre'])->name(name: 'adminSearchByGenre'); // Route
97     Route::get(uri: '/books/edit/{bookID}', action: [AdminController::class, 'edit'])->name(name: 'books.edit'); // Route to show the edit form (AJA
98     Route::put(uri: '/books/update/{book}', action: [AdminController::class, 'update'])->name(name: 'books.update'); // Route to update the book det
99     Route::post(uri: '/books/add', action: [AdminController::class, 'store'])->name(name: 'books.add'); // Route to add a book
100    Route::get(uri: '/books/add', action: [AdminController::class, 'addBookSuccess'])->name(name: 'books.index'); // Route to show success prompt
101    Route::post(uri: '/books/delete', action: [AdminController::class, 'deleteBooks'])->name(name: 'books.delete'); // Route to delete a book
102 });
```

- These routes are for admin users to manage books, users, and reviews. Admins can view dashboards, search for users and books, manage reviews, and handle user violations. They can also add, update, and delete books, with specific routes for editing and managing book details. The routes ensure admins can control all aspects of the website's content and user behavior.

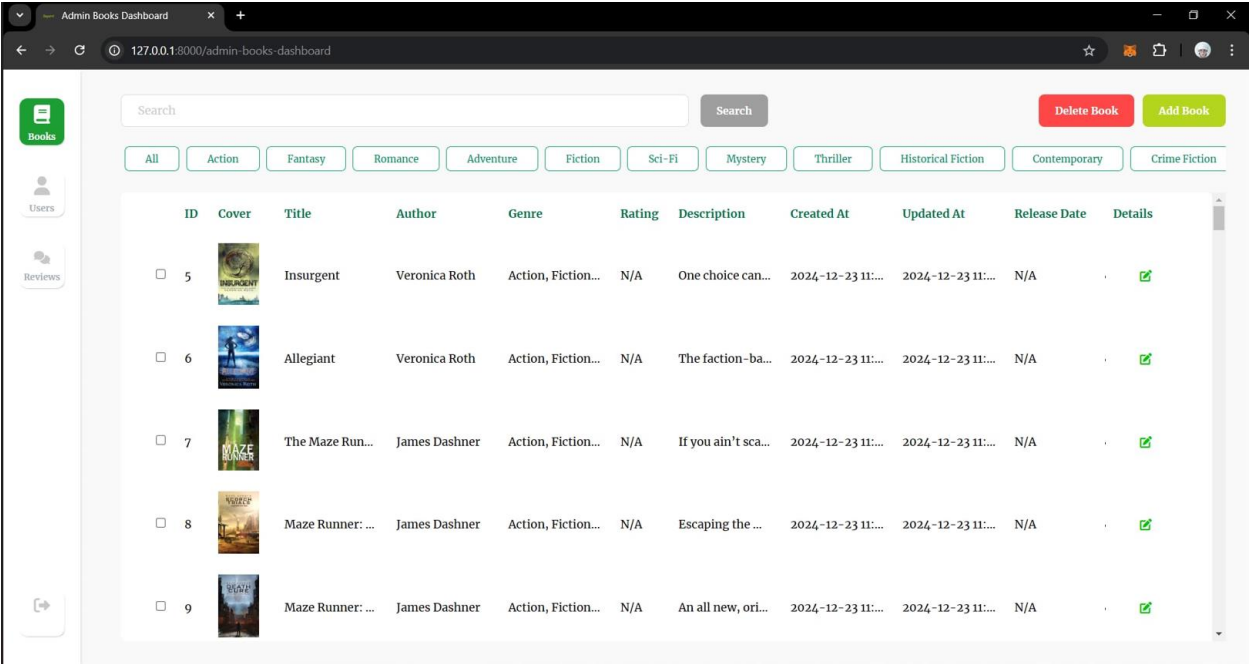
Part 2: Make an account registration and login page.

NOTE: This account registration and login page is for the admin side, since the one for the user is already shown in the laboratory 6. Thank You!

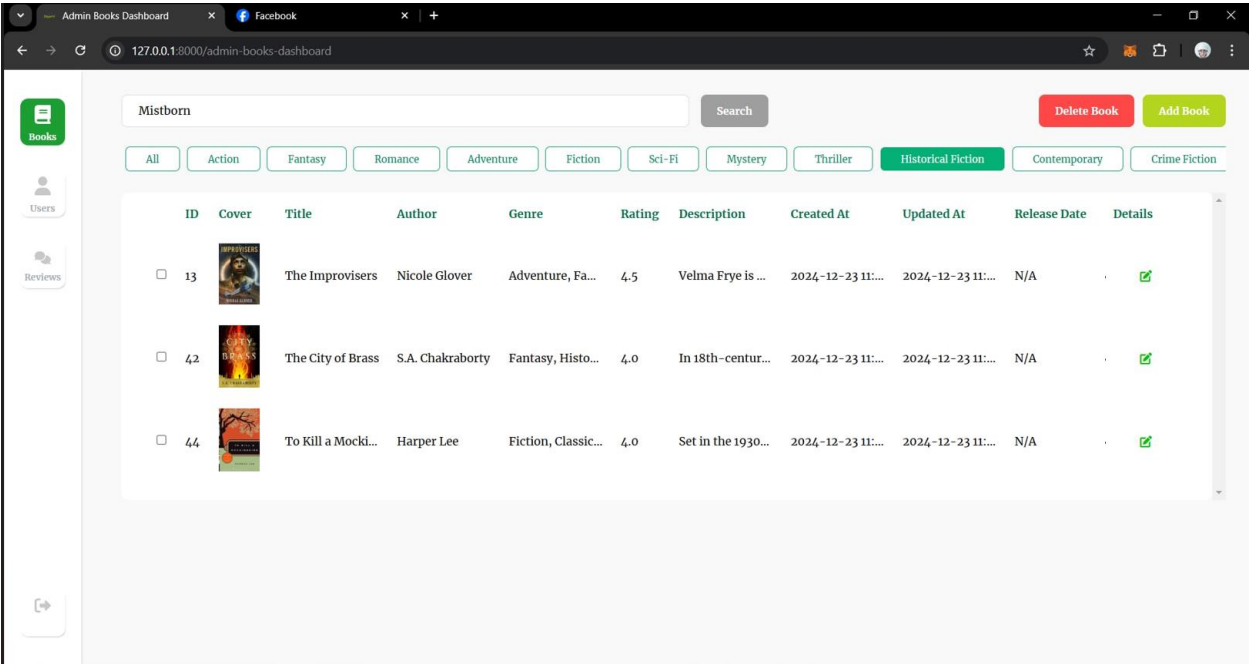
Admin Side



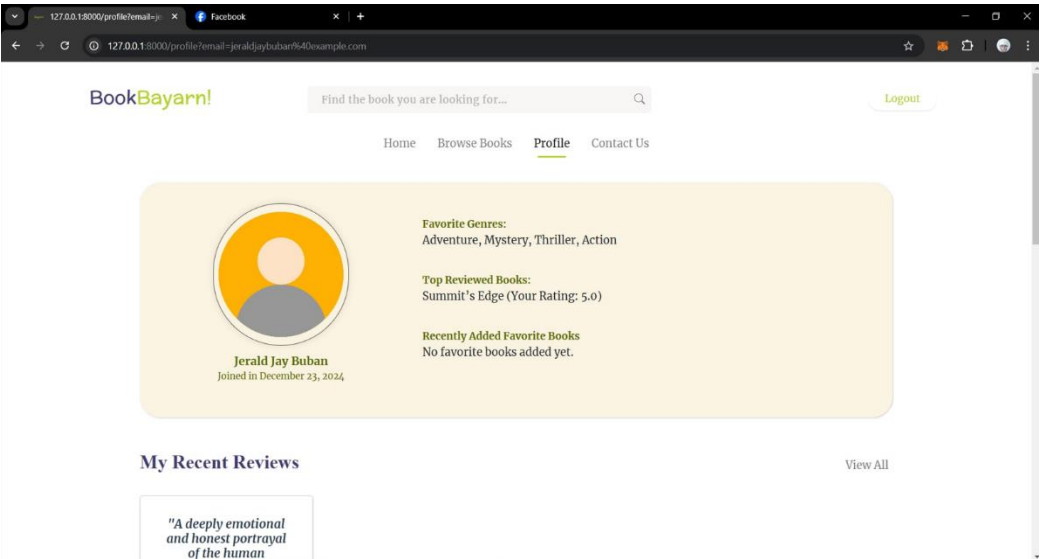
- The admin has a set email and password.



- After the admin login his/her account, they will be redirected to this view which is the admin books dashboard where they can see all the information's about a book.



- The admin can also search books here.



- Authenticated users will be redirected to their profile, which has some of their information's.

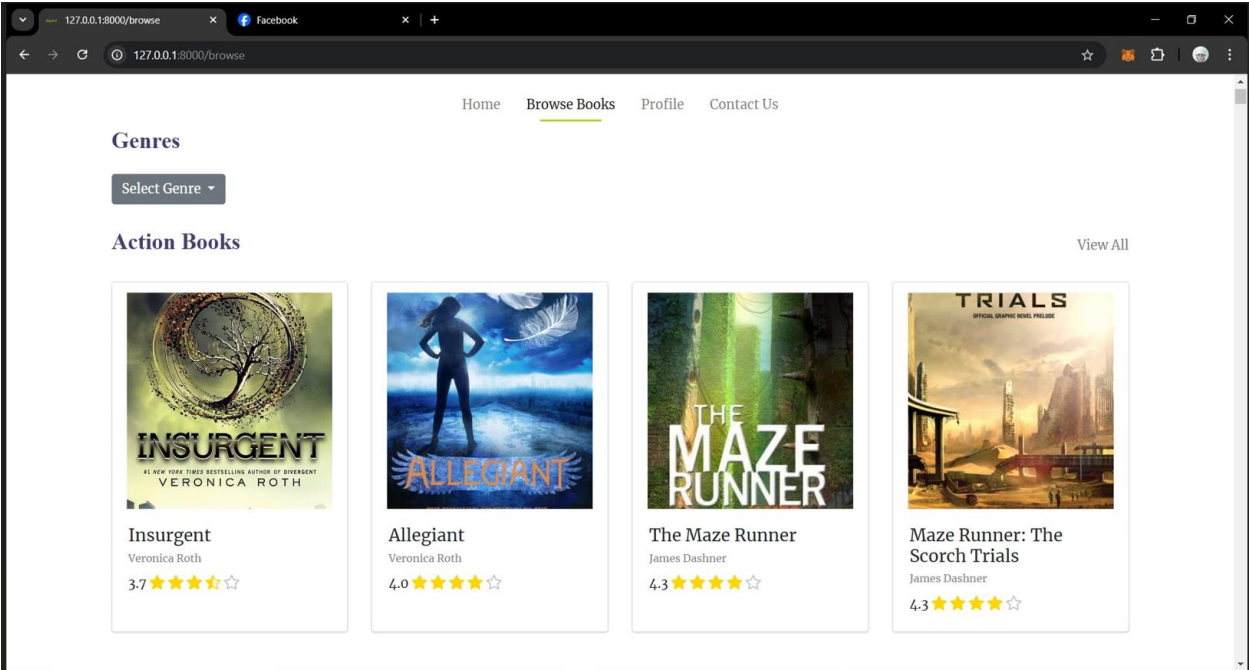
Part 3: Populate your pages using data fetched from a database.

Books Table in the Database

		bookID	title	author	genre	description	cover	created_at	updated_at	release_date
☐	Click the drop-down arrow to toggle column's visibility.		5 Insurgent	Veronica Roth	Action, Fiction, Fantasy, Sci-Fi, Romance	One choice can transform you—or it can destroy you...	assets/covers/Insurgent.jpg	2024-12-23 11:44:56	2024-12-23 11:44:56	NULL
☐	Edit Copy Delete		6 Allegiant	Veronica Roth	Action, Fiction, Fantasy, Sci-Fi, Romance	The faction-based society that Tris Prior once bel...	assets/covers/Allegiant.jpg	2024-12-23 11:44:56	2024-12-23 11:44:56	NULL
☐	Edit Edit Copy Delete		7 The Maze Runner	James Dashner	Action, Fiction, Fantasy, Sci-Fi, Thriller	If you ain't scared, you ain't human. When Thomas ...	assets/covers/The Maze Runner.jpg	2024-12-23 11:44:56	2024-12-23 11:44:56	NULL
☐	Edit Edit Copy Delete		8 Maze Runner: The Scorch Trials	James Dashner	Action, Fiction, Fantasy, Sci-Fi, Thriller	Escaping the Maze was only the beginning. The worl...	assets/covers/Scorch Trials.jpg	2024-12-23 11:44:56	2024-12-23 11:44:56	NULL
☐	Edit Edit Copy Delete		9 Maze Runner: The Death Cure	James Dashner	Action, Fiction, Fantasy, Sci-Fi, Thriller	An all new, original Maze Runner story set directl...	assets/covers/Death Cure.jpg	2024-12-23 11:44:56	2024-12-23 11:44:56	NULL
☐	Edit Edit Copy Delete		10 The Lightning Thief	Rick Riordan	Action, Adventure, Fantasy, Fiction	Percy Jackson is a good kid, but he can't seem to ...	assets/covers/The Lightning Thief.jpg	2024-12-23 11:44:56	2024-12-23 11:44:56	NULL
☐	Edit Edit Copy Delete		11 Heist Royale	Kayvion Lewi	Action, Mystery, Thriller, Fiction, Romance, Fanta...	The high-stakes sequel to Thieves' Gambit, for fan...	assets/covers/Heist Royale.jpg	2024-12-23 11:44:56	2024-12-23 11:44:56	NULL
☐	Edit Edit Copy Delete		12 Killing Floor	Lee Child	Action, Mystery, Thriller, Fiction, Romance, Fanta...	Ex-military policeman Jack Reacher is a drifter. H...	assets/covers/Killing Floor.jpg	2024-12-23 11:44:56	2024-12-23 11:44:56	NULL
☐	Edit Edit Copy Delete		13 The Improvisers	Nicole Glover	Adventure, Fantasy, Historical Fiction, Fiction, M...	Velma Frye is many things. A pilot, a former bootl...	assets/covers/The Improvisers.jpg	2024-12-23 11:44:56	2024-12-23 11:44:56	NULL
☐	Edit Edit Copy Delete		14 A Tale Told By Traitors	Renee Dugan	Adventure, Fantasy, Romance, Fiction	L'IONYRA VARA is not who she seems. ...	assets/covers/A Tale Told By Traitors.jpg	2024-12-23 11:44:56	2024-12-23 11:44:56	NULL
☐	Edit Edit Copy Delete		15 The Treasure Hunters Club	Tom Ryan	Adventure, Mystery, Thriller, Fiction	For nearly a century, people have ventured to the ...	assets/covers/The Treasure Hunters Club.jpg	2024-12-23 11:44:56	2024-12-23 11:44:56	NULL
☐	Edit Edit Copy Delete					The apocalypse will be				

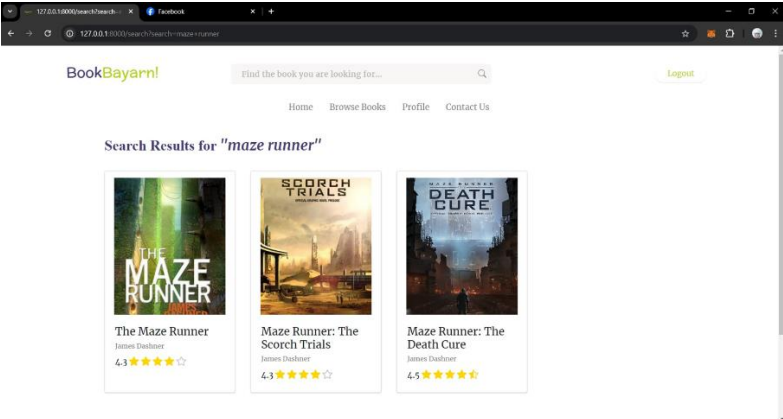
- It’s just the same with the one earlier, the only difference is that this one is from the database and the previous one is also from the database but already have a design.

Home Page

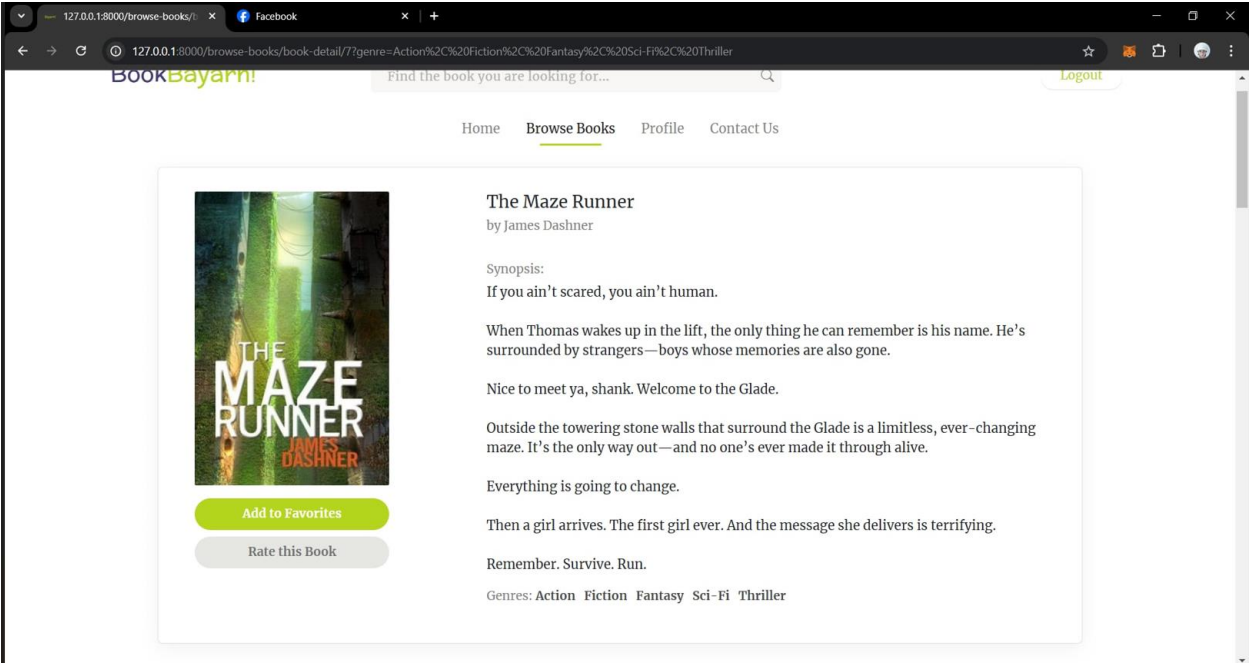


- This is the home page, where the user can already select the genre they want.

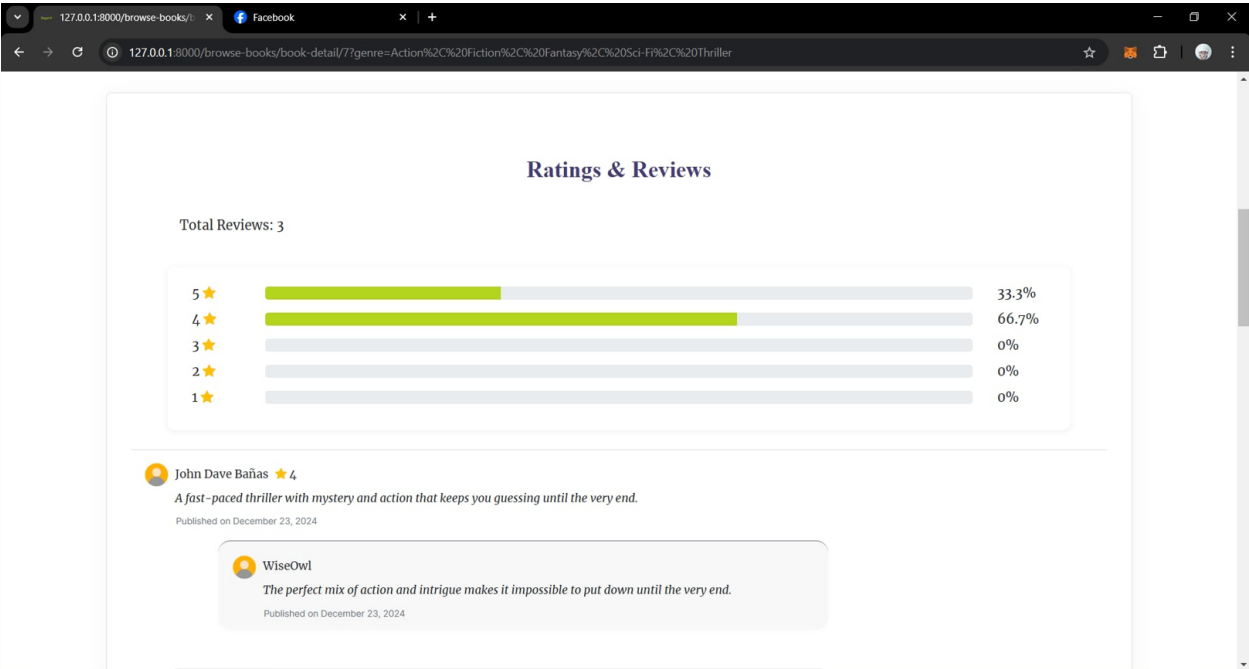
Search Feature



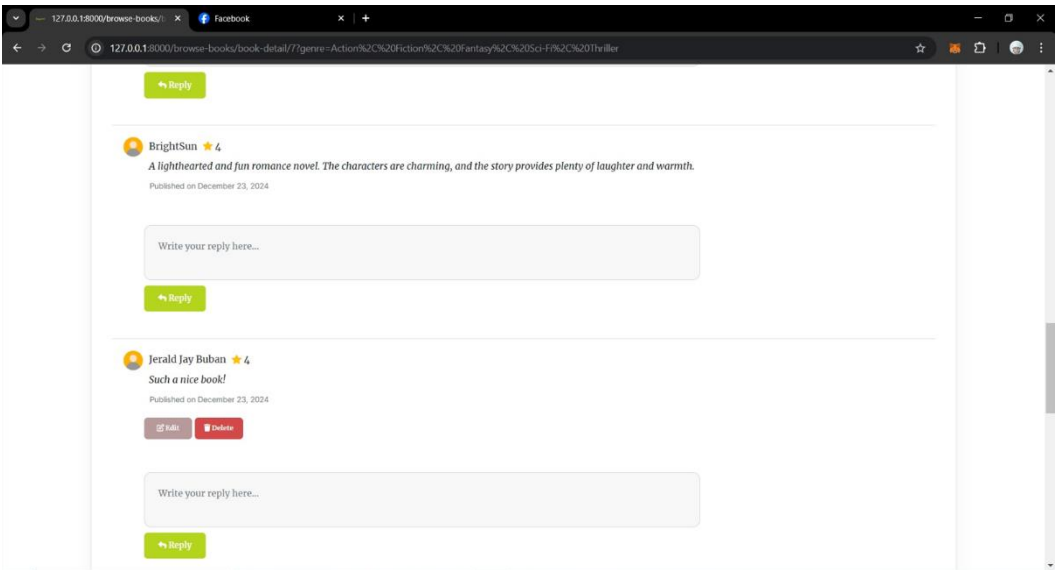
- Users can search the books they want based on the title, author, or genre.



- If the user clicked a book, he/she will be redirected to this page which contains the book title, book cover photo, author, description, and the option to add the book to their favorites or rate that book.

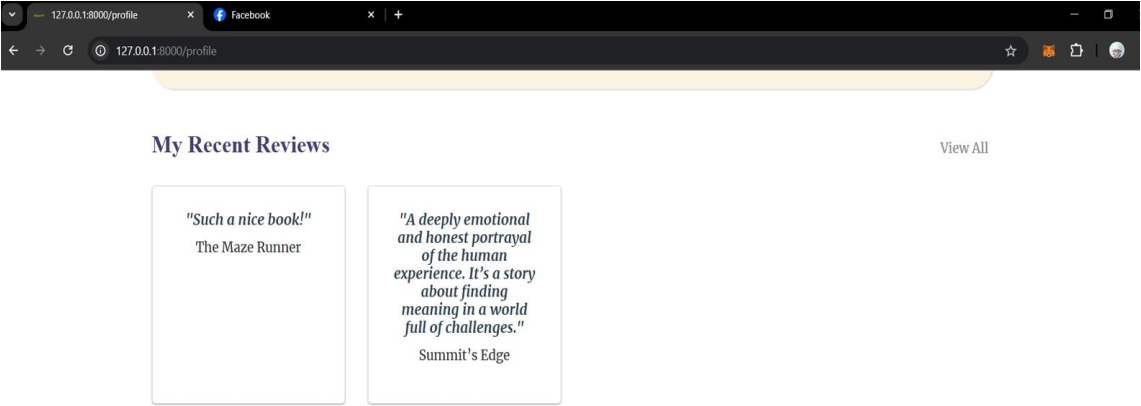
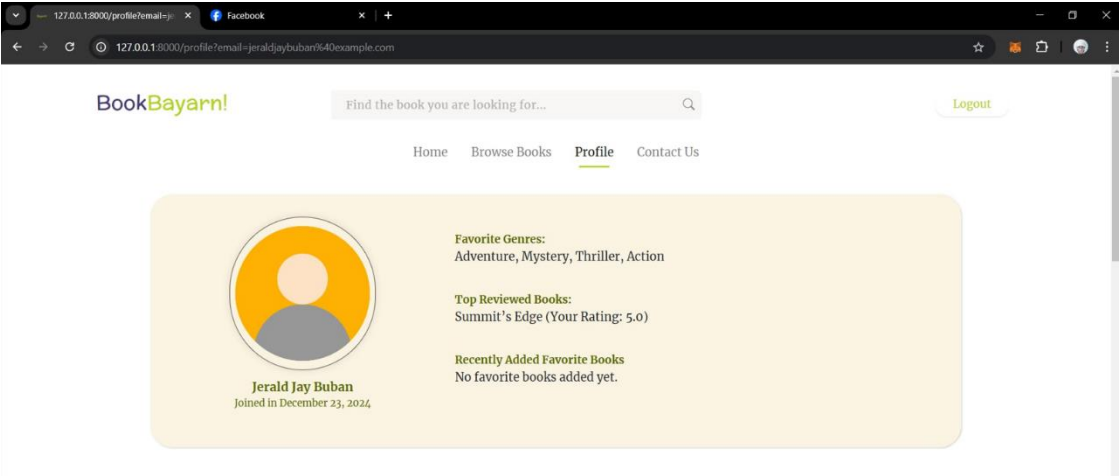


- And when the user scrolls down, they'll see the ratings and review of that book.

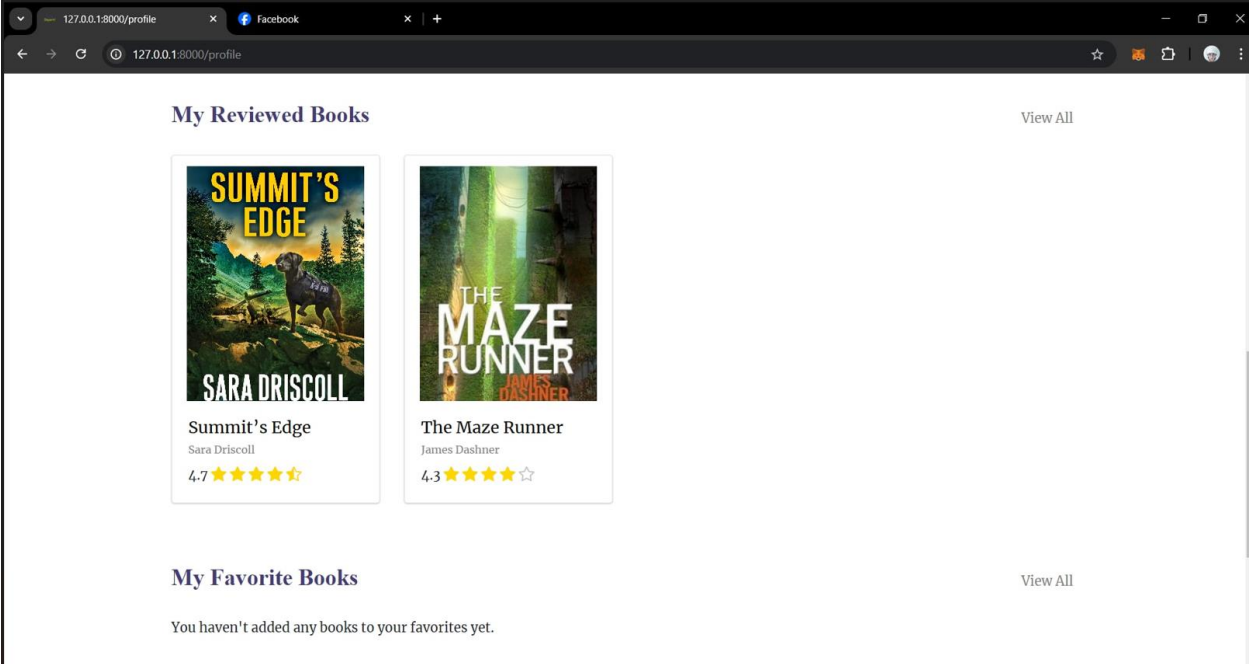


- And users can add a comment, reply to a comment, and edit or delete their comment/review.

Profile

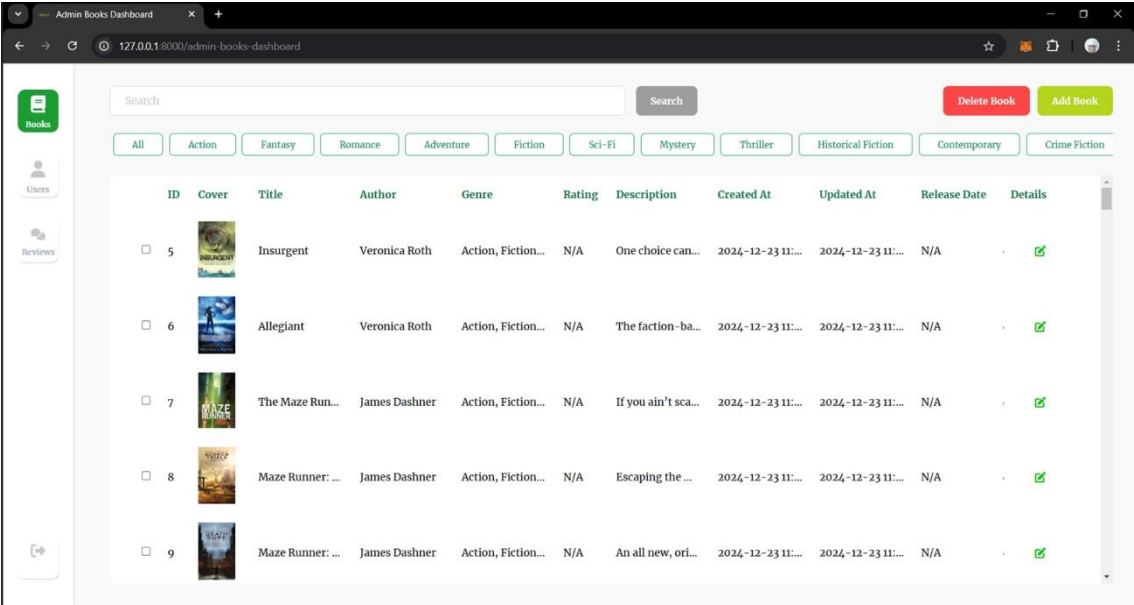


- In the profile, you can see your recent reviews.

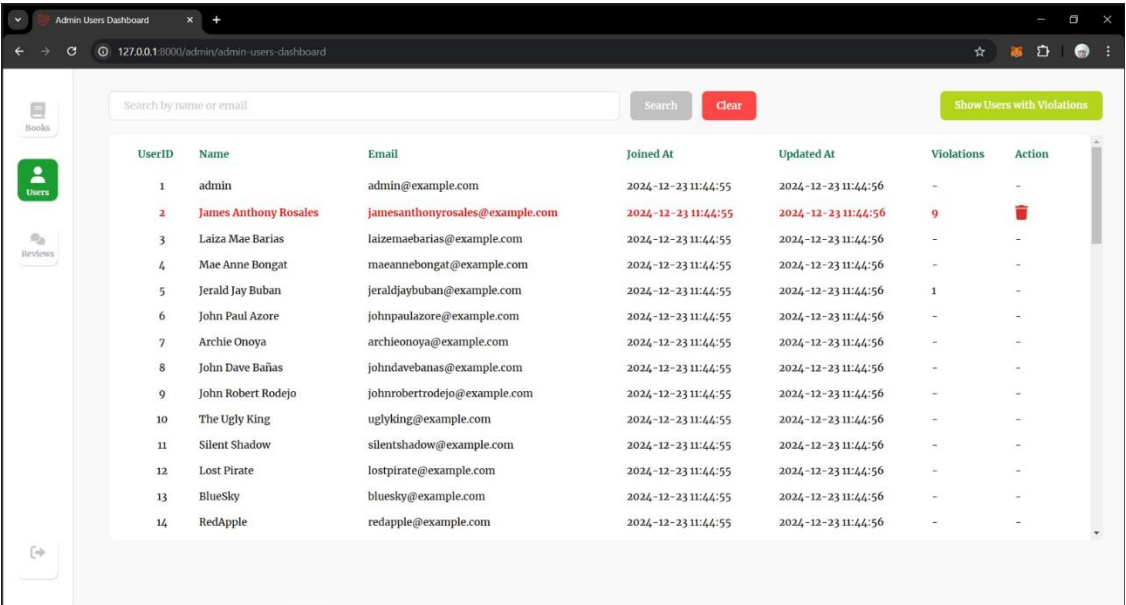


- You can also see here your reviewed books and your favorite books if ever you added one.

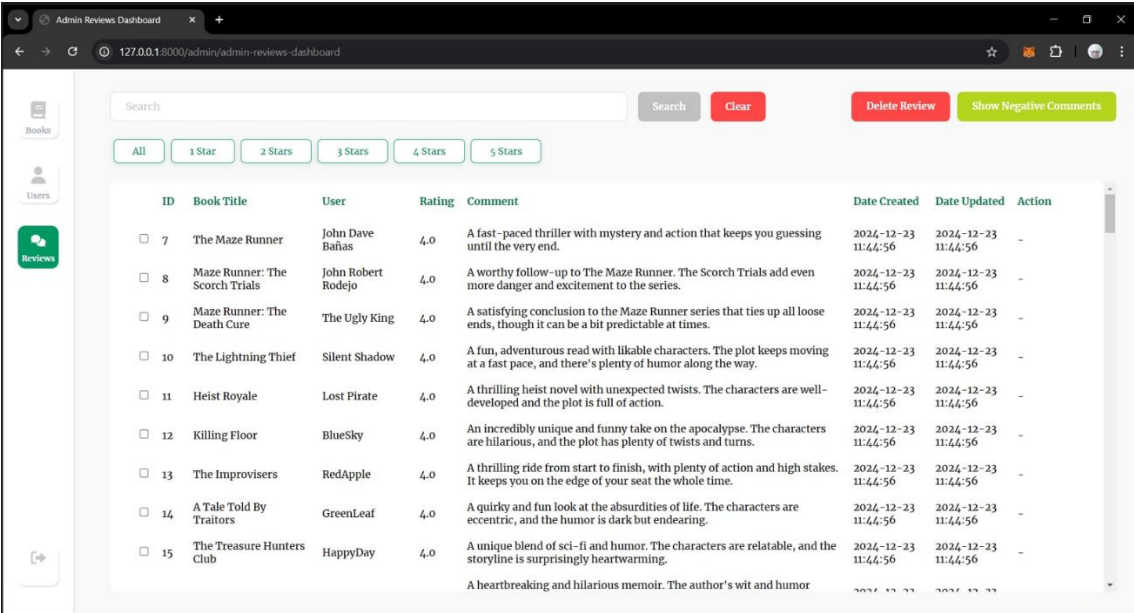
Admin Side Views



- In the books, it shows the lists of the books, which includes id, cover, title, author, genre, rating, description, created at, and updated at.

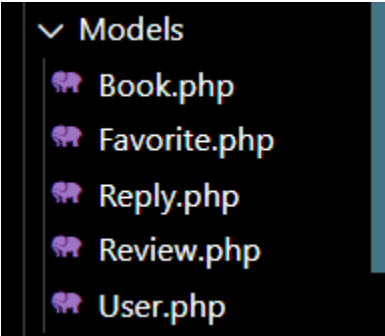


- In the users, it shows the list of the users that signed up and login and some of their information's, such as user id, name, email, time they've joined and updated, and violations.



- And in the reviews, it shows the list of the reviews/comments made by different users, it shows the ID, book title, name of the user that added that review, rating to it, the comment/review, and the date created and updated.

Models



- These are the models we have and used for this website.

Book Model

```
app > Models > Book.php > ...
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Factories\HasFactory;
7
8  class Book extends Model
9  {
10     use HasFactory;
11
12     protected $table = 'books';
13
14     public $primaryKey = 'bookID';
15
16     public $timestamps = true;
17
18     protected $fillable = ['title', 'author', 'genre', 'description', 'cover'];
19
20     // Define the one-to-many relationship with Review
21     public function reviews(): HasMany
22     {
23         return $this->hasMany(related: Review::class, foreignkey: 'bookID'); // The 'bookID' is the foreign key in the 'reviews' table
24     }
25
26
27     public function usersWhoFavorited(): BelongsToMany
28     {
29         return $this->belongsToMany(related: User::class, table: 'favorites', foreignPivotKey: 'book_id', relatedPiv... 'user_id');
30     }
31 }
```

- This Book model represents a book in the database, with properties like title, author, genre, description, and cover image. It has relationships with other models: a one-to-many relationship with reviews (a book can have many reviews) and a many-to-many relationship with users (who can favorite the book).

Reply Model

```
app > Models > Reply.php > ...
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7
8  3 references | 0 implementations
9  class Reply extends Model
10 {
11     use HasFactory;
12
13     // Define the table name (optional if you follow Laravel's conventions)
14     0 references
15     protected $table = 'replies';
16
17     // Mass assignable attributes
18     0 references
19     protected $fillable = [
20         'review_id', // Foreign key to the review
21         'user_id',   // Foreign key to the user
22         'comment',   // Reply content
23     ];
24
25     // A reply belongs to a review
26     0 references | 0 overrides
27     public function review(): BelongsTo
28     {
29         return $this->belongsTo(related: Review::class, foreignKey: 'review_id');
30     }
31
32     // A reply belongs to a user
33     0 references | 0 overrides
34     public function user(): BelongsTo
35     {
36         return $this->belongsTo(related: User::class);
37     }
38 }
```

- This Reply model represents a reply to a review. It contains attributes like review_id, user_id, and comment. The model defines relationships where each reply belongs to a specific review and a user.

Review Model

```
app > Models > Review.php > Review
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Factories\HasFactory;
7
8  19 references | 0 implementations
9  class Review extends Model
10 {
11     use HasFactory;
12
13     0 references
14     protected $table = 'reviews';
15
16     0 references
17     protected $primaryKey = 'reviewID'; // Change this if your primary key is not 'id'
18
19     0 references
20     public $timestamps = true;
21
22     // Define the inverse of the one-to-many relationship with Book
23     0 references
24     protected $fillable = ['userID', 'bookID', 'rating', 'comment'];
25
26     // Define the relationship for replies
27
28     0 references | 0 overrides
29     public function user(): BelongsTo
30     {
31         return $this->belongsTo(related: User::class, foreignKey: 'userID');
32     }
33
34     0 references | 0 overrides
35     public function replies(): HasMany
36     {
37         return $this->hasMany(related: Reply::class, foreignKey: 'review_id'); // 'review_id' is the foreign key in the replies table
38     }
39
40     0 references | 0 overrides
41     public function book(): BelongsTo
42     {
43         return $this->belongsTo(related: Book::class, foreignKey: 'bookID');
44     }
45
46     0 references | 0 overrides
47     public function reviews(): HasMany
48     {
49         return $this->hasMany(related: Review::class, foreignKey: 'bookID');
50     }
51 }
```

- The Review model represents a review for a book, with attributes like userID, bookID, rating, and comment. It defines relationships where each review belongs to a user and a book, and can have multiple replies.

User Model

```
app > Models > User.php > ...
1  <?php
2
3  namespace App\Models;
4
5  // use Illuminate\Contracts\Auth\MustVerifyEmail;
6  use Illuminate\Database\Eloquent\Factories\HasFactory;
7  use Illuminate\Foundation\Auth\User as Authenticatable;
8  use Illuminate\Notifications\Notifiable;
9
10 17 references | 0 implementations
11 class User extends Authenticatable
12 {
13     /** @use HasFactory<\Database\Factories\UserFactory> */
14     use HasFactory, Notifiable;
15
16     /**
17      * The attributes that are mass assignable.
18      *
19      * @var array<int, string>
20      */
21     0 references
22     protected $fillable = [
23         'name',
24         'email',
25         'password',
26         'profile_picture',
27     ];
28
29     /**
30      * The attributes that should be hidden for serialization.
31      *
32      * @var array<int, string>
33      */
34     0 references
35     protected $hidden = [
36         'password',
37         'remember_token',
38     ];
39
40     /**
41      * Get the attributes that should be cast.
42      *
43      * @return array<string, string>
44      */
45     0 references | 0 overrides
46     protected function casts(): array
47     {
48         return [
49             'email_verified_at' => 'datetime',
50             'password' => 'hashed',
51         ];
52
53     0 references | 0 overrides
54     public function reviews(): HasMany
55     {
56         return $this->hasMany(related: Review::class, foreignKey: 'userID');
57     }
58
59     0 references | 0 overrides
60     public function favoriteBooks(): BelongsToMany
61     {
62         return $this->belongsToMany(related: Book::class, table: 'favorites', foreignPivotKey: 'user_id', relatedPiv... 'book_id')->withTimestamps();
63     }
64 }
```

- The User model represents a user of the website and it includes attributes like name, email, and password. It defines relationships with reviews (a user can have many reviews) and favorite books (a user can have many books marked as favorites). The model also handles password hiding for security, as well as casting specific attributes like email_verified_at to a date and password to a hashed value for security purposes.